

Reinforcement learning: a primer

Yash Palan

August 14, 2026

Chapter 1

RL basics

Reinforcement Learning (RL) is one the most common techniques used in modern day artificial intelligence (AI). However, it is which is far away from standard machine learning techniques, partially since it solves a completely different problem. The aim of this writeup is to give a brief summary / crash course to the readers on the same, by introducing the basic mathematical notation, as well the latest algorithms, without going into too much depth, that a course would take but still giving the necessary pre-requisites to understand papers and how to code based on those papers.

1.1 Problem Setup for ML

The problem that RL tries to solve is generally that of navigation of an agent in an unknown external environment, so basically an agent who interacts with the environment and learns to take a particular action based on information that it receives from the environment. However, this can happen in multiple different ways, which is taken about more in the standard RL literature. Here, we just focus on setting up the notation that is used in the literature.

Problem: Consider the example of an agent (say a robot) who is interacting with the environment, which maybe be modelled using some model. The agent can stay in some of the states of the environment $s \in \mathcal{S}$ ($\mathcal{S}$ is the set of all possible states), and choose to take an action a from all possible actions $\mathcal{A}$, to switch to another state. Once an action has been taken, the agent receives a *feedback/reward* $r \in \mathbb{R}$ from the environment.

Goal: The goal of the agent is to maximize the reward.

Now, having introduced the basic problem, we introduce some definitions

Definitions

1. **Value function** $V(s)$: This represents the expected total (including future) rewards the agent can get by being in a state s . So it is basically the value of the state s . Answers the question “How good is it to be in this state?”.
2. **Q function** $Q(a, s)$: This represents the expected total (including future) rewards the agent can get by taking action a when in state s .
3. **Policy** $\pi_\theta(s)$:
 - A strategy or rulebook used in reinforcement learning that maps environmental states to specific actions and helps in selection of the best action from a given state.

- Can be Stochastic ($P(a|s)$, probability distribution) or deterministic ($f(a, s)$ deterministic function or lookup table).
4. **Episode:** This refers to the sequence of state and action pair that the agent takes with time, eg. $s_0, a_1, s_1, a_2, s_2, \dots, a_T, s_T$. This is also called a *trial* or *trajectory*.
 5. **Model-based:** Rely on the model of the environment; either the model is known or the algorithm learns it explicitly.
 6. **Model-free:** No dependency on the model during learning.
 7. **On-policy:** Use the deterministic outcomes or samples from the target policy to train the algorithm.
 8. **Off-policy:** Training on a distribution of transitions or episodes produced by a different behavior policy rather than that produced by the target policy.

1.2 Mathematical definitions

1. **Future Rewards** from time t ,

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \quad (1.1)$$

where γ is the decay factor for future rewards.

2. **Value function:**

$$V^\pi(s) = \mathbb{E}_\pi[G_t | S_t = s] = \mathbb{E}_\pi[G_0 | S_0 = s] \quad (1.2)$$

Note that by $\mathbb{E}_\pi$, we just mean that the whole average is done with policy π (as in the action selection). As an example, one can expand the above as

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^{k+1} R_{t+k+1} | S_t = s \right] \quad (1.3)$$

$$= \mathbb{E}_\pi \left[R_{t+1} + \gamma \sum_{k=0}^{\infty} \gamma^k R_{t+k+2} | S_t = s \right] \quad (1.4)$$

$$= \sum_a \pi(a|s) \sum_{a', s'} p(a', s' | a, s) r_{a'} + \sum_a \pi(a|s) \sum_{a', s'} p(a', s' | a, s) \gamma \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+2} | S_{t+1} = s' \right] \quad (1.5)$$

$$= \sum_a \pi(a|s) \sum_{a', s'} p(a', s' | a, s) r_{a'} + \sum_a \pi(a|s) \sum_{a', s'} p(a', s' | a, s) \gamma \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t'+k+1} | S_{t'} = s' \right] \quad (1.6)$$

Remember that by the definition of the Value function

$$V^\pi(s') = \mathbb{E}_\pi[G_{t'} | S_{t'} = s'], \quad (1.7)$$

thus we get that

$$V^\pi(s) = \sum_a \pi(a|s) \sum_{a', s'} p(a', s' | a, s) [r_{a'} + \gamma V^\pi(s')]. \quad (1.8)$$

3. **Q function:** In a similar fashion as the value function, we can define the Q function as

$$Q^\pi(s) = \mathbb{E}_\pi [G_t | S_t = s, A_t = a] = \mathbb{E}_\pi [G_0 | S_0 = s, A_0 = a]. \quad (1.9)$$

Similarly, as before, we can write

$$Q^\pi(s) = \mathbb{E}_\pi \left[\sum_{k=0} \gamma^{k+1} R_{t+k+1} | S_t = s, A_t = a \right] \quad (1.10)$$

$$= \mathbb{E}_\pi \left[R_{t+1} + \gamma \sum_{k=0} \gamma^k R_{t+k+2} | S_t = s, A_t = a \right] \quad (1.11)$$

$$= \sum_{a', s'} p(a', s' | a, s) r_{a'} + \sum_{a', s'} p(s', a' | s, a) \gamma \mathbb{E}_\pi \left[\sum_{k=0} \gamma^k R_{t+k+2} | S_{t+1} = s', A_{t+1} = a' \right] \quad (1.12)$$

$$= \sum_{a', s'} p(a', s' | a, s) r_{a'} + \sum_{a', s'} p(a', s' | a, s) \gamma \mathbb{E}_\pi \left[\sum_{k=0} \gamma^k R_{t'+k+1} | S_{t'} = s', A_{t'} = a' \right] \quad (1.13)$$

$$= \sum_{a', s'} p(a', s' | a, s) [r_{a'} + \gamma Q^\pi(s', a')] \quad (1.14)$$

It is easy to note that

$$p(s', a' | s, a) = p(s' | s, a) \pi(a' | s'), \quad (1.15)$$

to get us that

$$Q^\pi(s) = \sum_{a', s'} p(s' | s, a) \pi(a' | s') [r_{a'} + \gamma Q^\pi(s', a')] \quad (1.16)$$

$$= \sum_{s'} p(s' | s, a) \left[\sum_{a'} \pi(a' | s') r_{a'} + \sum_{a'} \pi(a' | s') \gamma Q^\pi(s', a') \right] \quad (1.17)$$

Using the fact that

$$V^\pi(s) = \sum_{a'} \pi(a' | s') Q^\pi(s', a') \quad (1.18)$$

and defining the total reward for a state as

$$r_s = \sum_{a'} r_{a'} \pi(a' | s') \quad (1.19)$$

we can rewrite the above equation as

$$Q^\pi(s) = \sum_{a', s'} p(s' | s, a) \pi(a' | s') [r_{a'} + \gamma Q^\pi(s', a')] \quad (1.20)$$

$$= \sum_{s'} p(s' | s, a) [r_{s'} + \gamma V^\pi(s')]. \quad (1.21)$$

Note that there is an abuse of notation for r where we assume that the subscript informs us of what it actually means. The above equation is the **Bellman equation for the Q function**.

4. **Weng's transition kernel** ($P(s', r | s, a)$): Probability that we go from state-action s, a to state s' with reward r . Note that

$$\sum_r P(s', r | s, a) = p(s' | s, a) \quad (1.22)$$

Using the above, we can rewrite the Bellmann's equation for the Q function as

$$Q^\pi(s) = \sum_{s',r} P(s',r|s,a) [r + \gamma V^\pi(s')] \quad (1.23)$$

Chapter 2

Proximal Policy optimisation

Proximal policy optimisation is an RL method which is an example of a gradient based on-policy technique. To understand what it does and how it works, we first give a brief introduction of Gradient based policy methods and then how this comes out of the same [1, 2].

2.1 Policy Gradient based methods

Policy gradient based techniques are methods that utilize finding the gradient of the Policy for finding the best policy, i.e. a policy that maximized the reward function $J(\theta)$ [empty citation]

$$J(\theta) = V^\pi(s) \quad (2.1)$$

where $V^\pi(s)$ is the value function of the state s under the policy π . Note that the above is for **episodic RL** (RL with discrete actions), however, the same works for **continuous RL** as well. Then, one can write

$$J(\theta) = \sum_{a \in \mathcal{A}} \pi_\theta(a|s) Q^\pi(a, s) \quad (2.2)$$

$$\text{where } V^\pi(s) = \sum_{a \in \mathcal{A}} \pi_\theta(a|s) Q^\pi(a, s) \quad (2.3)$$

Then, one can maximize the reward function by computing the gradient w.r.t. the optimisation parameters represented by θ ,

$$\nabla_\theta J(\theta) = \sum_{s \in \mathcal{S}} d^\pi(s) \sum_{a \in \mathcal{A}} \nabla_\theta \pi_\theta(a|s) Q^\pi(a, s) \quad (2.4)$$

where $d^\pi(s)$ represents a **stationary distribution**.

Proof of the Gradient Theorem

Now, we must note that equation 2.4 is not obvious, since $Q^\pi(a, s)$ also depends on the parameters θ via the policy π . Thus, we note that

$$\nabla_\theta V^\pi(s) = \sum_{a \in \mathcal{A}} [\nabla_\theta \pi_\theta(a|s) Q^\pi(a, s) + \pi_\theta(a|s) \nabla_\theta Q^\pi(a, s)] \quad (2.5)$$

Using the Bellmann equation (eq. 1.23)

$$\nabla Q^\pi(a, s) = \nabla_\theta \left[\sum_{s', r} P(s', r|s, a) [r + \gamma V^\pi(s')] \right] \quad (2.6)$$

$$= \sum_{s', r} \gamma P(s', r|s, a) \nabla_\theta V^\pi(s') \quad (2.7)$$

We can do this since $P(s', r|s, a)$ does not depend on the Policy.

Note: In principle we can link $P(s', r|s, a)$ to the policy but this is what most textbooks assume.

Thus, we now have that

$$\nabla Q^\pi(a, s) = \sum_{s'} \gamma p(s'|s, a) \nabla_\theta V^\pi(s'). \quad (2.8)$$

Using the above, we can now rewrite the gradient for the value function as

$$\nabla_\theta V^\pi(s) = \sum_{a \in \mathcal{A}} \left[\nabla_\theta \pi_\theta(a|s) Q^\pi(a, s) + \pi_\theta(a|s) \sum_{s'} \gamma p(s'|s, a) \nabla_\theta V^\pi(s') \right] \quad (2.9)$$

This effectively gives us a recursion relation for $\nabla V_\theta^\pi(s)$. To simplify, we just rewrite

$$\phi(s) = \sum_{a \in \mathcal{A}} \nabla_\theta \pi_\theta(a|s) Q^\pi(a, s) \quad (2.10)$$

$$\Pr(s \rightarrow x, 1, \pi) = \sum_{a \in \mathcal{A}} \pi(a|s) p(x|s, a). \quad (2.11)$$

Note that $\Pr(s \rightarrow x, 1, \pi)$ represents the probability of going from state s to x in one 1 step under policy π . Similarly we can consider the probability $\Pr(s \rightarrow x, k, \pi)$ of going from state s to x in one k steps under policy π . By induction, we can show that

$$\nabla_\theta V^\pi(s) = \sum_x \left(\sum_{k=0}^{\infty} \gamma^k \Pr(s \rightarrow x, k, \pi) \right) \sum_a \nabla_\theta \pi(a|s) Q^\pi(x, a) \quad (2.12)$$

Defining

$$\eta(x) = \left(\sum_{k=0}^{\infty} \gamma^k \Pr(s \rightarrow x, k, \pi) \right), \quad (2.13)$$

we get that

$$\nabla_\theta J(\theta) = \sum_s \eta(s) \sum_a \nabla_\theta \pi(a|s) Q^\pi(s, a) \quad (2.14)$$

$$= \left(\sum_s \eta(s) \right) \sum_s \frac{\eta(s)}{\sum_s \eta(s)} \sum_a \nabla_\theta \pi(a|s) Q^\pi(s, a) \quad (2.15)$$

$$\propto \sum_s d^\pi(s) \sum_a \nabla_\theta \pi(a|s) Q^\pi(s, a) \quad (2.16)$$

What the gradient theorem implies is that all that we need to maximize the reward function is to maximize the policy π_θ . How to do so is what we will see in the next sections, and where the various type

of RL models differ.

2.2 Monte Carlo based Gradient Policy optimisation

2.2.1 REINFORCE

The above now gives us an idea for the first **policy gradient based RL algorithm**. The reason the policy gradient theorem works is because we need a proportionality for gradient descent, since the proportionality factor can be absorbed in the step size α of the gradient descent optimizer. Thus, we can effectively say that

$$\nabla_{\theta} J(\theta) \propto \sum_s d^{\pi}(s) \sum_a \nabla_{\theta} \pi(a|s) Q^{\pi}(s, a) \quad (2.17)$$

$$= \mathbb{E}_{\pi} \left[\sum_a \nabla_{\theta} \pi(a|S_t) Q^{\pi}(S_t, a) \right] \quad (2.18)$$

where $\mathbb{E}_{\pi} \equiv \mathbb{E}_{S_t \sim d^{\pi}(s)}$. In a similar fashion, we can also write

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi} \left[\sum_a \pi(a|S_t) Q^{\pi}(S_t, a) \frac{\nabla_{\theta} \pi(a|S_t)}{\pi(a|S_t)} \right] \quad (2.19)$$

$$= \mathbb{E}_{\pi} \left[Q^{\pi}(S_t, A_t) \frac{\nabla_{\theta} \pi(A_t|S_t)}{\pi(A_t|S_t)} \right] \quad (2.20)$$

$$= \mathbb{E}_{\pi} [Q^{\pi}(S_t, A_t) \nabla_{\theta} \log \pi_{\theta}(A_t|S_t)] \quad (2.21)$$

where now $\mathbb{E}_{\pi} \equiv \mathbb{E}_{S_t \sim d^{\pi}, A_t \sim \pi_{\theta}}$. We note here

$$\mathbb{E}_{S_t \sim d^{\pi}, A_t \sim \pi_{\theta}} \equiv \sum_{s_0, a_0} \sum_{s_1, a_1} \dots \sum_{s_T, a_T} \sum_{t=0}^T [Q^{\pi}(S_t, A_t) \nabla \log \pi_{\theta}(A_{t,n}|S_{t,n})] \quad (2.22)$$

$$= \mathbb{E}_{\pi} [Q^{\pi}(S_t, A_t) \nabla \log \pi_{\theta}(A_{t,n}|S_{t,n})] \quad (2.23)$$

so the expectation value is taken over the sequence $s_0, a_0, r_1, \dots, s_{T-1}, a_{T-1}, r_T$. We also note that for later times, we can define the Q value as

$$Q^{\pi}(S_t, A_t) = \gamma^t G_t, \quad (2.24)$$

to account for the delay in the path. In the case the decay rate $\gamma = 1$, we get back our original non-decay version of the gradient theorem.

Reason for equivalence

Objective as a value function:

$$J(\theta) = v_{\pi}(s_0) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t R_{t+1} \mid S_0 = s_0 \right]$$

Weng's unrolled gradient (sum over states, weighted by total discounted visitation):

$$\nabla_{\theta} J(\theta) = \sum_{x \in \mathcal{S}} \eta(x) \phi(x), \quad \eta(x) = \sum_{k=0}^{\infty} \rho^{\pi}(s_0 \rightarrow x, k), \quad \phi(x) = \sum_a \nabla_{\theta} \pi_{\theta}(a|x) Q^{\pi}(x, a)$$

Normalizing η into a proper distribution μ :

$$\mu(x) = \frac{\eta(x)}{\sum_{x'} \eta(x')} \implies \nabla_{\theta} J(\theta) \propto \mathbb{E}_{S \sim \mu} [\phi(S)] = \mathbb{E}_{\pi} [q_{\pi}(S, A) \nabla_{\theta} \ln \pi_{\theta}(A|S)]$$

This last expression is exactly **Image 1**: a single-sample expectation, with all the “sum over t ” structure absorbed into the distribution μ that S is drawn from.

Discounted state-visitation distribution (the proper version of μ , with explicit normalization):

$$d_{\gamma}(s | s_0) = (1 - \gamma) \sum_{t=0}^{\infty} \gamma^t P(S_t = s | S_0 = s_0)$$

Identity linking a trajectory-sum to a single-state expectation, for any function f :

$$\mathbb{E}_{S \sim d_{\gamma}} [f(S)] = (1 - \gamma) \sum_{t=0}^{\infty} \gamma^t \mathbb{E}[f(S_t) | S_0 = s_0]$$

Rearranged — this is the explicit correspondence between the two forms:

$$\sum_{t=0}^{\infty} \gamma^t \mathbb{E}[f(S_t) | S_0 = s_0] = \frac{1}{1 - \gamma} \mathbb{E}_{S \sim d_{\gamma}} [f(S)]$$

Applied to the two images directly:

$$\underbrace{\mathbb{E}_{\pi_{\theta}} \left[\sum_{t=0}^T \nabla_{\theta} \ln \pi_{\theta}(A_t | S_t) \sum_{\tau=t}^T \gamma^{\tau} R_{\tau} \middle| S_0 = s_0 \right]}_{\text{Image 2 — explicit sum over } t \text{ along one trajectory}} \propto \underbrace{\mathbb{E}_{S \sim \mu, A \sim \pi} \left[G_t \frac{\nabla_{\theta} \pi_{\theta}(A|S)}{\pi_{\theta}(A|S)} \right]}_{\text{Image 1 — single sample from the occupancy measure}}$$

Summary of the resolution: Image 2 is the literal, unsimplified sum over every step t of a trajectory rooted at a fixed s_0 . Image 1 is that same quantity after recognizing the sum over t can be rewritten as a single expectation over one state S drawn from the (discounted) state-occupancy distribution μ (equivalently d_{γ}) induced by π . They are the same gradient, related by the identity above — Image 2 is what you actually compute from a rollout; Image 1 is the compressed form used in proofs.

In fact, we can simplify the above even more by using the definition of the q function to get that

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi} [G_t \nabla_{\theta} \log \pi_{\theta}(A_t | S_t)] \quad (2.25)$$

where G_t is the delayed total reward at time t . Note that we have now converted an expectation value over the distribution into a time series based task using the above expression.

Sutton algorithm

Now, if we think about the above expression, we note that, in principle, one would need to average over multiple different. However, one can then consider the **Stochastic Gradient descent (SGD)** strategy where we need to consider only one of the paths. Thus, under SGD, we can write the update equation to be as follows

Update equation

$$\theta \rightarrow \theta + \alpha [G_t \nabla \log \pi_\theta(A_t|S_t)] \quad (2.26)$$

where $[G_t \log \pi_\theta(A_t|S_t)]$ is sampled by using the current policy π . Thus, we find G_t by simulating the evolution using the policy π and then compute the gradient.

The algorithm for the same is thus given as follows

Algorithm

Input: A differentiable polict $\pi_\theta(a|s)$.

Initialize: $\theta \in \mathbb{R}^d$

Repeat:

 Generate an episode $s_0, a_0, r_1, s_1, \dots, s_{T-1}, a_{T-1}, r_T$ following π_θ .

 For each step of the episode $t = 0, \dots, T-1$

$G \rightarrow$ return from step t

$\theta \rightarrow \theta + \alpha \gamma^t G_t \nabla_\theta \log(\pi_\theta(A_t|S_t))$

Wikipedia algorithm

A more natural update equation for the same is as below [3]

Update equation

Consider N trajectories with different starting points $s_{0,i}$ where $i = 1, \dots, N$. Then, the update equation is

$$\theta \rightarrow \theta - \alpha \frac{1}{N} \sum_{n=1}^N \sum_{t=0}^T [\gamma^t G_{t,n} \nabla \log \pi_\theta(A_{t,n}|S_{t,n})] \quad (2.27)$$

where $[G_{t,n} \log \pi_\theta(A_{t,n}|S_{t,n})]$ is computed by getting the sequence $S_{0,n}, A_{0,n}, R_{1,n}, \dots, S_{T-1,n}, A_{T-1,n}, R_{T,n}$ using the policy π_θ . This is our standard Gradient descent that we know of.

The algorithm for the same is thus given as follows

Algorithm

Input: A differentiable polict $\pi_\theta(a|s)$.

Initialize: $\theta \in \mathbb{R}^d$

Choose N.

Repeat:

 for $n = 1, \dots, N$:

 Generate an episode $s_{0,n}, a_{0,n}, r_{1,n}, s_{1,n}, \dots, s_{T-1,n}, a_{T-1,n}, r_{T,n}$ following π_θ .

 For each step of the episode $t = 0, \dots, T-1$, compute

$G_{t,n} \rightarrow \sum_{\tau=t}^T \gamma^\tau r_{\tau,n}$: return from step t

$\theta \rightarrow \theta + \alpha \frac{1}{N} \sum_{n=1}^N \sum_{t=0}^T [\gamma^t G_{t,n} \nabla \log \pi_\theta(A_{t,n}|S_{t,n})]$

For simplicity, we will now work with the sequential definition of the expectation value, since that is easier to work with.

2.2.2 REINFORCE with baseline

However, it turns out that the variance seen in the evolution for the REINFORCE algorithm is too high. So, we need a way to reduce the variance in the evolution. This can be easily achieved by considering the subtraction of a **baseline** function $b(s)$ which only depends on the state s [2]. So, the Gradient is then given by

$$\nabla_{\theta} J(\theta) = \sum_{s \in \mathcal{S}} d^{\pi}(s) \sum_{a \in \mathcal{A}} \nabla_{\theta} \pi_{\theta}(a|s) (Q^{\pi}(a, s) - b(s)) \quad (2.28)$$

The above is correct since

$$\sum_{s \in \mathcal{S}} d^{\pi}(s) \sum_{a \in \mathcal{A}} b(s) \nabla_{\theta} \pi_{\theta}(a|s) = \sum_{s \in \mathcal{S}} d^{\pi}(s) b(s) \nabla_{\theta} \sum_{a \in \mathcal{A}} \pi_{\theta}(a|s) \quad (2.29)$$

But $\pi(a|s)$ is a conditional probability, and thus $\sum_{a \in \mathcal{A}} \pi_{\theta}(a|s) = 1$, implying that

$$\sum_{s \in \mathcal{S}} d^{\pi}(s) \sum_{a \in \mathcal{A}} b(s) \nabla_{\theta} \pi_{\theta}(a|s) = 0, \quad (2.30)$$

thus giving us the original result. In a fashion similar to the previous derivation, we also note that

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi} [(Q^{\pi}(S_t, A_t) - b(S_t)) \nabla_{\theta} \log \pi_{\theta}(A_t|S_t)] \quad (2.31)$$

where $\mathbb{E}_{\pi} \equiv \mathbb{E}_{S_t \sim d^{\pi}, A_t \sim \pi_{\theta}}$.

Alternative derivation

Let us now consider an alternative derivation for the same. We follow the calculation as shown in [4]. The idea is now to use the sequential definition of the expectation value to derive the same result as before. To do so, we start with the following expression

$$\tilde{J}(\theta) = \mathbb{E}_{s_0, a_0} \mathbb{E}_{a_{1:T}, s_{1:T}} \sum_t [\{Q^{\pi}(S_t, A_t) - b(S_t)\} \nabla_{\theta} \log \pi_{\theta}(A_t|S_t)]. \quad (2.32)$$

Let us just focus on the term

$$\mathbb{E}_{s_0, a_0} \mathbb{E}_{a_{1:T}, s_{1:T}} \sum_{t=0}^T [b(S_t) \nabla_{\theta} \log \pi_{\theta}(A_t|S_t)]. \quad (2.33)$$

In fact, we just consider for a fixed t ,

$$\mathbb{E}_{s_0, a_0} \mathbb{E}_{a_{1:T}, s_{1:T}} [b(S_t) \nabla_{\theta} \log \pi_{\theta}(A_t|S_t)] = \mathbb{E}_{a_{0:t}, s_{0:t}} b(S_t) \mathbb{E}_{a_{1:t+1}, s_{1:t+1}} [\nabla_{\theta} \log \pi_{\theta}(A_t|S_t)] \quad (2.34)$$

However, note that $\nabla_{\theta} \log \pi_{\theta}(A_t|S_t)$ is an effective constant for the variation of times $t+1 : T$, since A_t and S_t are fixed then. Also the expectation value of a constant is nothing but that constant, implying that

$$\mathbb{E}_{a_{t+1:T}, s_{t+1:T}} [\nabla_{\theta} \log \pi_{\theta}(A_t|S_t)] = \nabla_{\theta} \log \pi_{\theta}(A_t|S_t) \quad (2.35)$$

Similarly, we can also say that

$$\mathbb{E}_{a_{0:t-1}, s_{0:t-1}} [\nabla_{\theta} \log \pi_{\theta}(A_t|S_t)] = \nabla_{\theta} \log \pi_{\theta}(A_t|S_t) \quad (2.36)$$

Thus, we are left with

$$\mathbb{E}_{s_0, a_0} \mathbb{E}_{a_{1:T}, s_{1:T}} [b(S_t) \nabla_{\theta} \log \pi_{\theta}(A_t | S_t)] \quad (2.37)$$

$$= \mathbb{E}_{a_t, s_t} b(s_t) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) = \sum_{s_t \in \mathcal{S}, a_t \in \mathcal{A}} b(s_t) P(a_t, s_t) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \quad (2.38)$$

$$= \sum_{s_t \in \mathcal{S}, a_t \in \mathcal{A}} b(s_t) \pi_{\theta}(a_t | s_t) P(s_t) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \quad (2.39)$$

$$\text{as } P(a_t, s_t) = \pi_{\theta}(a_t | s_t) P(s_t) \quad (2.39)$$

$$= \sum_{s_t \in \mathcal{S}, a_t \in \mathcal{A}} b(s_t) P(s_t) \nabla_{\theta} \pi_{\theta}(a_t | s_t) \quad (2.40)$$

$$= \sum_{s_t \in \mathcal{S}} b(s_t) P(s_t) \nabla_{\theta} \sum_{a_t \in \mathcal{A}} \pi_{\theta}(a_t | s_t) \quad (2.41)$$

$$= \sum_{s_t \in \mathcal{S}} b(s_t) P(s_t) \nabla_{\theta} 1 \quad (2.42)$$

$$= 0 \quad (2.43)$$

Thus we see that for arbitrary t , the contribution from the $b(s_t)$ term is 0. Thus, we get that

$$\mathbb{E}_{s_0, a_0} \mathbb{E}_{a_{1:T}, s_{1:T}} \sum_{t=0}^T [b(S_t) \nabla_{\theta} \log \pi_{\theta}(A_t | S_t)] = 0 \quad (2.44)$$

and thus

$$\tilde{J}(\theta) = \mathbb{E}_{s_0, a_0} \mathbb{E}_{a_{1:T}, s_{1:T}} \sum_t [\{Q^{\pi}(S_t, A_t)\} \nabla_{\theta} \log \pi_{\theta}(A_t | S_t)] = J(\theta). \quad (2.45)$$

Thus proved

One may now question if the above reduction truly helps, as in if this truly reduces the variance. To show that this does indeed reduce the variance we need to compute

$$\text{Var}_{\pi} \left(\sum_t [\{Q^{\pi}(S_t, A_t) - b(S_t)\} \nabla_{\theta} \log \pi_{\theta}(A_t | S_t)] \right) \quad (2.46)$$

Proof of the variance reduction

So, the task is to find $\text{var}(\sum_{t=0}^T X_t)$ where

$$X_t = \{Q^{\pi}(S_t, A_t) - b(S_t)\} \nabla_{\theta} \log \pi_{\theta}(A_t | S_t). \quad (2.47)$$

To simplify, we make the following assumptions [4]

1. $\text{Cov}(X_t, X_{t'}) = 0$, so that we can write $\text{Var}(\sum_t X_t) = \sum_t \text{Var}(X_t)$. Eventhough this may not be true, it still helps us understand how the baseline works.
2. The second approximation that we take is the independence of the terms $\{Q^{\pi}(S_t, A_t)\}$ and $\nabla_{\theta} \log \pi_{\theta}(A_t | S_t)$.

We now see that

$$\text{Var}\left(\sum_{t=0}^T X_t\right) = \sum_{t=0}^T \text{Var}(X_t) \quad (2.48)$$

$$= \sum_{t=0}^T \mathbb{E}(X_t^2) - \mathbb{E}^2 \quad (2.49)$$

However, we have shown before that

$$\mathbb{E} \left[\sum_{t=0}^T \{Q^\pi(S_t, A_t) - b(S_t)\} \nabla_\theta \log \pi_\theta(A_t|S_t) \right] = \mathbb{E} \left[\sum_{t=0}^T \{Q^\pi(S_t, A_t)\} \nabla_\theta \log \pi_\theta(A_t|S_t) \right]. \quad (2.50)$$

Thus we can ignore the $\sum_{t=0}^T \mathbb{E}_\pi(X_t)^2$ term and focus on the former term. Using the **indepence of the individual terms**, we can say that

$$\sum_{t=0}^T \mathbb{E}(X_t^2) = \sum_{t=0}^T \mathbb{E}_\pi(\{Q^\pi(S_t, A_t) - b(S_t)\}^2) \mathbb{E}_\pi(\{\nabla_\theta \log \pi_\theta(A_t|S_t)\}^2) \quad (2.51)$$

Thus, we see that the variance does indeed reduce when $b(S_t) \neq 0$.

2.2.3 Advantage function

We have seen briefly that the introduction of the baseline reduces the variance. In principle one can choose it to be a constant $b(S_t) = \text{const}$. However, the choice of a baseline would be better if we make it state dependent. A very simple example of such a function is the Value function ($V(S_t)$). Thus, we can now define a new function called the **Advantage function** ($A(s_t, a_t)$) as

$$A(s_t, a_t) = Q(s_t, a_t) - V(s_t). \quad (2.52)$$

In general, the advantage function will depend on the policy $\pi(s_t, a_t)$, so we get that $A_\pi(s_t, a_t)$. The gradient theorem is thus updated as

$$J(\theta) = \mathbb{E}_\pi \left[\sum_t A^\pi(S_t, A_t) \nabla_\theta \log \pi_\theta(A_t|S_t) \right]. \quad (2.53)$$

2.3 Action-Critic models

Having introduced the idea of the Advantage function, we can now look at the idea of actor-critic RL models.

Actor-critic model

The actor-critic algorithm (AC) is a family of reinforcement learning (RL) algorithms that combine policy-based RL algorithms such as policy gradient methods, and value-based RL algorithms such as value iteration, Q-learning, SARSA, and TD learning. [5].

Let us now understand the two basic components of these models

1. **Actor** (π_θ):

- The actor is the basically the one who decides what action must be taken next. So, in some sense it is just another name for our **Policy** $\pi_\theta(s_t, a_t)$.
- **Input:** States
- **Output:** Probability distribution of the actions.

2. Critic:

- The critic is the network that decides the value of the current state.
- A simple example of the critic is the Value function $V(S_t)$ or the Q function $Q(a_t, s_t)$.

To understand how an actor-critic method works, let us consider an example of the same.

Actor-critic example

Let us consider an example of a discrete RL model where the the next choice is chosen based on the Policy $\pi_\theta(s_t, a_t)$. Next, for the critic, we consider learning the Q function $Q_\pi(a_t, s_t)$. So, we have two neural nets, one learning the Policy and the second the Q function, with one affecting the learning of the other.

We now look at the equation

Let us now consider the algorithm for the same

Actor critic algorithm

Input: Policy $\pi_\theta(A|s)$ and $V_w(s)$
Parameters: θ, w and Step sizes $\alpha_\theta > 0, \alpha_w > 0$
Initialize: Parameters θ, w .
 $I = 1$ Repeat forever:
 Initialise the first state s .
 while s is not terminal:
 $A \sim \pi_\theta(\cdot|s)$.
 Take action A and see the next state and reward S', R .
 $\delta = R + \gamma V_w(s') - V_w(s)$. If s is terminal, then $V_w(s') = 0$. $w \rightarrow w + \alpha_w \delta I \nabla_w V_w(s)$
 $\theta \rightarrow \theta + \alpha_\theta \delta I \nabla_\theta \pi_\theta(a|s)$
 $I \rightarrow \gamma I$ $S \rightarrow S'$

Here what has been combined is the TD(1) method for the critic and the Gradient Policy method for the actor.

In fact, current literature has generalised the possible gradient theorem

$$J(\theta) = \mathbb{E}_\pi \left[\sum_{t=0}^T \nabla_\theta \pi_\theta(a_t|s_t) \Psi_t | S_0 = s_0 \right] \quad (2.54)$$

where Ψ_t can have the following values

- $\Psi_t = \gamma^t \sum_{\tau=t}^T \gamma^{\tau-t} R_\tau$: REINFORCE algorithm
- $\Psi_t = \gamma^t \sum_{\tau=t}^T \gamma^{\tau-t} R_\tau - b(s_t)$: REINFORCE with baseline algorithm
- $\Psi_t = \gamma^t (R_t + \gamma V^\pi(s_{t+1}) - V^\pi(s_t))$: Time Difference (TD)-1 learning
- $\gamma^t Q^\pi(a_t, s_t)$
- $\gamma^t A^\pi(a_t, s_t)$: Advantage Actor-Critic (A2C)
- $\Psi_t = \gamma^t (\sum_{k=0}^{n-1} R_{t+k} + \gamma^n V^\pi(s_{t+n}) - V^\pi(s_t))$: TD(n) learning

The above shows the variation in the different types of actor-critic models.

2.4 Off vs On policy

2.4.1 On Policy

Methods whose training samples are collected according to the target policy — the very same policy that we try to optimize for are called as **on policy methods**[1]. Eg- REINFORCE, vanilla gradient.

2.4.2 Off Policy

In Off-policy learning we seek to learn a value function for a target policy π , given data due to a different behavior policy b . In the prediction case, both policies are static and given, and we seek to learn either state values $\tilde{V} \approx V$ or action values $\tilde{Q} \approx Q$. In the control case, action values are learned, and both policies typically change during learning— π being the greedy policy with respect to $\tilde{Q}$, and b being something more exploratory such as the greedy policy with respect to $\tilde{Q}$ [2].

Off policy methods result in several additional advantages[1]:

- The off-policy approach does not require full trajectories and can reuse any past episodes (“experience replay”) for much better sample efficiency.
- The sample collection follows a behavior policy different from the target policy, bringing better exploration.

Now let’s see how off-policy policy gradient is computed. The behavior policy for collecting samples is a known policy (predefined just like a hyperparameter), labelled as $\beta(a|s)$. The objective function sums up the reward over the state distribution defined by this behavior policy:

$$J(\theta) = \sum_{s \in \mathcal{S}} d^\beta(s) \sum_{a \in \mathcal{A}} Q_{a,s}^\pi \pi_\theta(a, s) = \mathbb{E}_{s \sim \beta} \left[\sum_{a \in \mathcal{A}} Q_{a,s}^\pi \pi_\theta(a, s) \right]. \quad (2.55)$$

Notice here that we sample the states s from the **stationary distribution of the policy β** and NOT that of π_θ . In this way the two differ. We recall here that

$$d^\beta(s) = \lim_{t \rightarrow \infty} P(S_t = s | s_0, \beta). \quad (2.56)$$

Now to get an expression similar to the previous on-policy case, we again consider

$$\nabla_\theta J(\theta) = \nabla_\theta \mathbb{E}_{s \sim \beta} \left[\sum_{a \in \mathcal{A}} Q_{a,s}^\pi \pi_\theta(a, s) \right] \quad (2.57)$$

$$= \mathbb{E}_{s \sim \beta} \left[\sum_{a \in \mathcal{A}} \nabla (Q_{a,s}^\pi \pi_\theta(a, s) + Q_{a,s}^\pi \nabla(\pi_\theta(a, s))) \right] \quad (2.58)$$

$$\approx \mathbb{E}_{s \sim \beta} \left[\sum_{a \in \mathcal{A}} Q_{a,s}^\pi \nabla(\pi_\theta(a, s)) \right]. \quad (2.59)$$

The former term is thrown away for two reasons

1. Generally computing $\nabla_\theta Q^\pi(a, s)$ is extremely hard.
2. In the paper [6], they claim that the solution set obtained from the approximate gradient is a super set of the true one, and is indeed the same for the tabular case (for functional case of Q^π , the two converge for small enough steps). Effectively, they say that since the gradient greedily changes the action probabilities for only one state. Thus, extrapolating from this, in practice, more generally a local representation for π suffices, since parameter updates influence only a small number of states. Similarly, in the non-tabular case, the claim holds if γ is small (the return is myopic), again because changes to the policy mostly affect the action-value function locally.

Thus, we can drop proceed with our derivation. Then

$$\nabla_{\theta} J(\theta) \approx \mathbb{E}_{s \sim \beta} \left[\sum_{a \in \mathcal{A}} Q_{a,s}^{\pi} \nabla(\pi_{\theta}(a, s)) \right] \quad (2.60)$$

$$\nabla_{\theta} J(\theta) \approx \mathbb{E}_{s \sim \beta} \left[\sum_{a \in \mathcal{A}} \beta(a|s) \frac{\pi(a|s)}{\beta(a|s)} Q^{\pi}(a, s) \frac{\nabla \pi_{\theta}(a|s)}{\pi_{\theta}(a|s)} \right] \quad (2.61)$$

$$\nabla_{\theta} J(\theta) \approx \mathbb{E}_{s \sim d^{\beta}, a \sim \beta(a)} \left[\frac{\pi(a|s)}{\beta(a|s)} Q^{\pi}(a, s) \frac{\nabla \pi_{\theta}(a|s)}{\pi_{\theta}(a|s)} \right]. \quad (2.62)$$

The result of this manipulation is that given a fixed policy β to sample our trajectories from, we can find the minimum of the true policy π_{θ} by computing the gradient in a fashion that does not need repeated sampling. This trick has effectively made sampling easier and less cumbersome since we can sample the trajectory and thus the Q function for each state-action pair beforehand, making our learning faster.

2.5 Proximal Policy Approximation (PPO)

We now have all the ingredients to understand the **Proximal Policy optimisation (PPO)** methodology of Reinforcement learning. PPO is an **On policy** optimisation strategy that was made by OpenAI [7] to improve training stability. This is an optimised version of the **Trust region policy optimisation (TRPO)** strategy, one which we will talk about later. It so happens that the TRPO strategy is hard to implement, but is something we would still like to do. To do so, PPO implements a **clipped surrogate objective**, while still retaining similar performance.

2.5.1 TRPO: Brief summary

However, before we do get to the main details, let us consider the reason for why we need it and so we briefly introduce the **Trust region policy Optimisation (TRPO)** method. The idea is that the reward function

$$J(\theta) = \mathbb{E}_{s \sim d^{\pi_{\theta_{\text{old}}}}, a \sim \beta(a)} \left[\frac{\pi(a|s)}{\beta(a|s)} A^{\pi}(a, s) \right] \quad (2.63)$$

where A is the advantage function. When training on policy, theoretically the policy for collecting data is same as the policy that we want to optimize. However, when rollout workers and optimizers are running in parallel asynchronously, the behavior policy can get stale. TRPO considers this subtle difference: It labels the behavior policy as $\pi_{\theta_{\text{old}}}(a|s)$ and thus the objective function becomes

$$J(\theta) = \mathbb{E}_{s \sim d^{\pi_{\theta_{\text{old}}}}, a \sim \pi_{\theta_{\text{old}}}} \left[\frac{\pi(a|s)}{\pi_{\theta_{\text{old}}}(a|s)} A^{\pi}(a, s) \right]. \quad (2.64)$$

TRPO thus aims to maximize the objective function subject to *trust region constraint* which enforces that the distance between the old and new policies, as measured by the KL divergence, be within parameter δ , i.e.

$$\mathcal{E}_{s \sim d^{\pi_{\theta_{\text{old}}}}, a \sim \pi_{\theta_{\text{old}}}} [D_{KL}(\pi_{\theta_{\text{old}}}(\cdot|s) || \pi_{\theta}(\cdot|s))] \leq \delta \quad (2.65)$$

2.5.2 PPO: actual policy

We note that despite the above idea being interesting, enforcing the trust region is hard during evolution. To make this simpler, PPO basically implements a **clipped surrogate objective** as a replacement for the TRPO constraint. We first define

$$r(\theta) = \frac{\pi_{\theta}}{\pi_{\theta_{\text{old}}}} \quad (2.66)$$

Then, we have that

$$J^{TRPO}(\theta) = \mathbb{E}[r(\theta)(a|s)A_{\theta_{\text{old}}}(a, s)] \quad (2.67)$$

while for PPO, we have

$$J^{CLIP}(\theta) = \mathbb{E}[\min(r(\theta)A_{\theta_{\text{old}}}(a|s), \text{clip}(r(\theta), 1 + \epsilon, 1 - \epsilon)A(a, s))] \quad (2.68)$$

where $\text{clip}(r(\theta), 1 + \epsilon, 1 - \epsilon)$ clips the ratio $r(\theta)$ to be no more than $1 + \epsilon$ less than $1 - \epsilon$.

In the PPO paper, the authors used a modified version of the reward function, which also took into account exploration by introducing the terms

$$\tilde{J}^{\text{CLIP}} = \mathbb{E}[J^{\text{CLIP}} - c_1(V_\theta(s) - V_{\text{target}})^2 + c_2 H(s, \pi_\theta(\cdot|s))] \quad (2.69)$$

where $H(s, \pi_\theta(\cdot|s))$ is the entropy term to encourage exploration. We will not discuss the same in much detail.

2.5.3 PPO: Algorithm

Having now introduced the idea behind PPO, we now look at the algorithm. We focus our attention on the **clip** policy. The algorithm is taken from OpenAI's writeup [8].

PPO Algorithm

1. **Input:** Initial policy parameters θ_0 , Initial value parameters ϕ_0
2. for $k = 0, 1, 2 \dots$ do:
3. Collect a set of trajectories $\mathcal{D}_k = \tau_i$ by running the policy $\pi_{\theta_k}(a|s)$ in the environment.
4. Compute the Rewards to go $R_t = \sum_{\tau=0}^T \gamma^\tau r_{t+\tau}$. In fact, one can also use the recursion relation

$$R_t = r_t + \gamma R_{t+1} \quad (2.70)$$

to make the calculation simple.

5. Compute the Advantage $A(a_t, s_t)$ (using any advantage estimate) based on the current Value function V_{ϕ_k} . One simple advantage estimate that can be used is **TD advantage**

$$A(a_t, s_t) = r_t + \gamma V_{t+1}(s_{t+1}) - V_t(s_t). \quad (2.71)$$

Or, one can also use the expression

$$A(a_t, s_t) = R_t - V_{s_t} \quad (2.72)$$

where we have replaced $Q^\pi(a_t, s_t) = R_t$ by the rewards to go function.

6. Update the policy by maximizing the PPO objective

$$\mathcal{L}(\theta) = \frac{1}{|\mathcal{D}_k|} \frac{1}{T} \sum_{\tau \in \mathcal{D}_k} \sum_{t=0}^T \left[\min \left(\frac{\pi_\theta(a|s)}{\pi_{\theta_{\text{old}}}(a|s)} A_{\theta_{\text{old}}}(a|s), \text{clip} \left(\frac{\pi_\theta(a|s)}{\pi_{\theta_{\text{old}}}(a|s)}, 1 + \epsilon, 1 - \epsilon \right) A_{\theta_{\text{old}}}(a, s) \right) \right] \quad (2.73)$$

by any optimisation technique (say SGD or Adam)

$$\theta_k \rightarrow \theta_k + \alpha \nabla_{\theta} \mathcal{L}(\theta) \quad (2.74)$$

7. Fit value function by **Regression** on mean squared error

$$\frac{1}{|\mathcal{D}_k T|} \sum_{\tau \in \mathcal{D}_k} \sum_{t=0}^T (V_{\phi}(s_t) - R_t)^2 \quad (2.75)$$

8. end

One may now ask how to compute the advantage function A from the value function V . For simplicity, we will use the **Time difference (TD) advantage**

$$A(a_t, s_t) = r_t - V(s_t) + \gamma V(s_{t+1}) \quad (2.76)$$

where r_t is the reward for the particular action a_t in state s_t . One can also refer to [9] for more details on the same.

Bibliography

- [1] Lilian Weng. *Policy Gradient Algorithms* — *lilianweng.github.io*. <https://lilianweng.github.io/posts/2018-04-08-policy-gradient/\#dpg>. [Accessed 08-08-2026].
- [2] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. Second. The MIT Press, 2018. URL: <http://incompleteideas.net/book/the-book-2nd.html>.
- [3] Wikipedia. *Policy gradient method* — *Wikipedia, The Free Encyclopedia*. <http://en.wikipedia.org/w/index.php?title=Policy\%20gradient\%20method&oldid=1358016730>. [Online; accessed 09-August-2026]. 2026.
- [4] *Going Deeper Into Reinforcement Learning: Fundamentals of Policy Gradients* — *danieltakeshi.github.io*. <https://danieltakeshi.github.io/2017/03/28/going-deeper-into-reinforcement-learning-fundamentals-of-policy-gradients/>. [Accessed 08-08-2026].
- [5] Wikipedia. *Actor-critic algorithm* — *Wikipedia, The Free Encyclopedia*. <http://en.wikipedia.org/w/index.php?title=Actor-critic\%20algorithm&oldid=1348373706>. [Online; accessed 09-August-2026]. 2026.
- [6] Thomas Degris, Martha White, and Richard S. Sutton. *Off-Policy Actor-Critic*. 2013. arXiv: 1205.4839 [cs.LG]. URL: <https://arxiv.org/abs/1205.4839>.
- [7] John Schulman et al. *Proximal Policy Optimization Algorithms*. 2017. arXiv: 1707.06347 [cs.LG]. URL: <https://arxiv.org/abs/1707.06347>.
- [8] *Proximal Policy Optimization — Spinning Up documentation* — *spinningup.openai.com*. <https://spinningup.openai.com/en/latest/algorithms/ppo.html\#id7>. [Accessed 13-08-2026].
- [9] Shivang Shrivastav. *Demystifying the Advantage Function in Reinforcement Learning* — *shivang-ahd.medium.com*. <https://shivang-ahd.medium.com/demystifying-the-advantage-function-in-reinforcement-learning-1c2b2a0d0daa>. [Accessed 13-08-2026].